

WEEK 4 LABORATORY — PUTTING THE TABLES TOGETHER

Merging queries · orphan keys · refresh

Session	Saturday, 5 September 2026 · four hours, in person
Outcome	Combine multiple business data files and explain the ETL process
Milestone	B2 — Cleaned and Documented Dataset (Weeks 3–4)
Due	Monday, 7 September 2026, 11:59 PM
Submit to	b2-cleaned-dataset in BDAT04-2026_T1
Next week	Preliminary Assessment, Saturday 12 September

One sentence to carry all day. Append adds rows.
Merge adds columns.
Everything in this laboratory is one of those two things, and every mistake you will make today is doing one when you meant the other.

0. Rebuild before you join

You cannot join tables that are not there. Every one of you has something to repair from last week before this laboratory can start.

1. Open your model and go to Power Query.
2. Delete every step you added in order to *look* at something — Group By, Keep Duplicates, Keep Errors, and any filter you added to see a count.
3. Keep only the changes you decided on. Rename each one so it says what it does.
4. Set the data type of every column you will join on. **A text key will not reliably match a numeric one.**
5. Refresh. Confirm your row count is what you expect before you go further.

Expected after a correct rebuild:

Query	Result
tblSales	10,566 rows / 9 columns
tblInventory	1,541 rows / 6 columns

tblEmployees	33 rows
---------------------	---------

Result: Confirmed.

Expected after a correct rebuild: tblSales 10,566 rows and 9 columns · tblInventory 1,541 rows and 6 columns · tblEmployees 33 rows. If your numbers differ, find out why before you merge anything.

1. Append — the July file

Operations has sent through July. It is a separate file, **KapeTayo_Sales_Jul2026.xlsx**, the same shape as your sales table with a new month in it. This is how a warehouse actually receives data: not once, but every month, forever.

1a. Load it

1. Get data → Excel workbook → **KapeTayo_Sales_Jul2026.xlsx** → select **tblSalesJul2026**.
2. Record the row count before you do anything else: **601 rows**

1b. Append it

1. Select your cleaned **tblSales** query.
2. Home → **Append queries** → choose the July query → OK.
3. Record the new row count: **11,167 rows**. Does it equal the two counts added together? **Yes. $10,566 + 601 = 11,167$.**

Now look at your columns. Count them.

You had nine. How many do you have now? **The original tblSales used PaymentMethod, while the July file used PaymentType. Since append matches columns by name, Power Query created two separate columns for the same type of information.**

Something has gone wrong, and it has gone wrong silently. Find it before you read on — the answer is in the column headers, not in the rows.

1c. What happened

Append matches columns **by name**. Any name that does not match on both sides becomes its own column, half full of nulls. Two columns now hold the same fact and neither one is complete.

1. Undo the append.
2. In the July query, rename the offending column so it matches yours exactly. Name the step: **Renamed to match tblSales**.
3. Append again. Confirm you are back to nine columns and the rows still add up.

Final result after appending again:

- **11,167 rows**
- **9 columns**

Write one sentence for your transformation notes explaining why you renamed it rather than deleting either column.

I renamed PaymentType to PaymentMethod so the July data would append into the existing sales field instead of creating two separate partially null columns

2. Merge — and the question it answers

A merge attaches columns from another table where a key matches. The useful part is not what matches. It is what does not.

2a. Join kinds

Join kind	What it keeps	Use
Left outer	All of mine; theirs where it matches	The normal join
Inner	Only rows that matched on both sides	Handle with care
Left anti	ONLY my rows that found no match	Your diagnostic
Full outer	Everything from both sides	Rare
Right outer	All of theirs	Rare
Right anti	Only their unmatched rows	Rare

Inner join is the dangerous one. It deletes every row that did not match and says nothing at all. If you use it, you must be able to state how many rows it removed — which means running the anti join first.

2b. The move of the day — left anti join

1. Home → **Merge queries** → as new.
2. Pick your two tables and click the joining column in each.
3. Join kind → **Left anti (rows only in first)** → OK.
4. What comes back is every row whose key does not exist in the other table. Count it.

Last week you asked: how many values are in this column? This week: how many of them are real?

3. The integrity lab

Six joins. Predict each one before you run it. One of the six comes back empty, and that is the most important result on the page.

Relationship	On	Predict	Orphan rows found
tblSales → tblBranches	BranchID		66
tblSales → tblProducts	ProductID		9
tblSales → tblCalendar	Date		771 returned
tblSales → tblCustomers	CustomerID		5047 returned
tblInventory → tblCalendar	Date		82
tblEmployees → tblBranches	BranchID		1

For each one that is not zero, write down the distinct offending values, not just the count. A number tells you how big the problem is; the values tell you what it is.

3a. The one that comes back empty

One relationship has 4,775 blanks and zero orphans. Write down which one, and then write two sentences on the difference between these:

Blank	Orphan
No value was recorded at all.	A value was recorded, and it points at something that does not exist.
A walk-in customer who never joined the loyalty programme.	A sale at a branch that is not in the branch table.
A fact about the business.	A fault in the data.

Missing is not broken. A student who deletes blanks because they look untidy has just removed 45% of this company's revenue.

4. What you do with an orphan

Four options. None of them is deleting it quietly.

	When	Example in this case
FIX	The value is a recognisable typo for something real	b01 is obviously B01
EXCLUDE	It genuinely is not part of the analysis	Report the count you removed
ESCALATE	You cannot tell, and guessing would be inventing data	P99 sold nine times and does not exist
ACCEPT	The value is real; the reference table is what is incomplete	B07 opened in July

Whichever you choose, it goes in the step name and in your notes.
A silent fix is the offence — not the fix itself.
Somebody reading your query in November has to be able to see the decision and disagree with it.

Two of the orphans you will find today are not faults at all. Work out which two before you touch anything.

Initial finding: 66 unmatched rows → FIX recognizable BranchID formatting issues → 26 B07 rows remain → ACCEPT B07

Decision:

FIX the recognizable formatting errors

ACCEPT the remaining B07 records

The recognizable BranchID formatting errors were fixed because their intended valid values could be identified without guessing. After the fixes, 26 unmatched rows remained, all with B07. These were accepted because the activity states that B07 opened in July, meaning the sales and employee records are valid while tblBranches is incomplete.

5. Parameters — stop typing the file path

If your query contains a path like `C:\Users\yourname\Downloads\...` then your model works on exactly one machine and can never refresh anywhere else. Fix it now; it takes four minutes.

1. Home → **Manage parameters** → New.
2. Name it **SourcePath**, type Text, and paste your folder path as the current value.
3. Open each query's Source step and replace the typed path with the parameter.
4. Change the parameter once and every query follows.

This is the difference between a file that works on your laptop and a model that works.

6. Query folding — in concept

Where it can, Power Query pushes your steps back to the source and lets the source do the work. Filter early and the source filters for you. Add a custom column first and it cannot — so every row is dragged across before anything is thrown away.

The rule for this course: remove rows and columns as early as you can; add clever things as late as you can. Step order is not only about readability. On a real dataset it is the difference between four seconds and four minutes.

7. The refresh

The course gate, from the syllabus: your model must refresh end to end from the supplied source files **without manual repair**. A report that cannot be refreshed is not a report — it is a spreadsheet that happened once.

1. Close and apply. Refresh the whole model from source.
2. Every query must complete. If any one fails, the milestone is not finished.
3. Then export your model: Model view → TMDL View → drag each table in → copy → save as `model_2026-09-05.tmdl` in your backups folder.

8. Two decisions, out loud, no notes

Before you leave, each of you will stand up for three minutes and defend two cleaning decisions from your own work. This is not a presentation and there are no slides.

Say	Meaning
What you did	Name the step and how many rows it touched
Why you were entitled to	The business reason, not the technical one
What was lost	Every cleaning step destroys something. Name it.

Next Saturday's preliminary assessment carries this as a graded criterion. Today is your rehearsal, and it is free.

B2 — what to submit

Item	Detail
Appended and merged queries	The case tables joined into an analysis-ready set

Transformation notes	Every decision, its reason, and what it discarded
Integrity report	Six relationships tested, orphans counted, a decision recorded for each
A successful full refresh	End to end from source, no manual repair
TMDL export	model_2026-09-05.tmdl in your backups folder
AI Use Log	Updated. Tier and purpose.

Build B2 on the ORIGINAL case file. Whatever file arrives during the session is for the session only — re-point your source before you leave the room, or you will submit a model that refers to a file nobody else has.

If you get stuck

Fifteen minutes, then ask. You are working in pairs today, so ask your partner first and me second. Tell me the section number, what you clicked, and what happened.

— Sir JB